Singly Linked List –
Reverse List Using Stack
and Time Complexity.

***Program*: –**

```
#include <stack>
….
void reverseListUsingStack(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        stack<Node *> s;
        Node *current = *head;
        while (current != nullptr)
        {
            s.push(current);
            current = current->next;
        }
        *head = s.top();
        s.pop();
        current = *head;
        while (!s.empty())
        {
            current->next = s.top();
            s.pop();
            current = current->next;
        }
        current->next = nullptr;
    }
}
```

```
…..

case 15:
        {
            reverseListUsingStack(&head);

        }
```

Program Analysis:

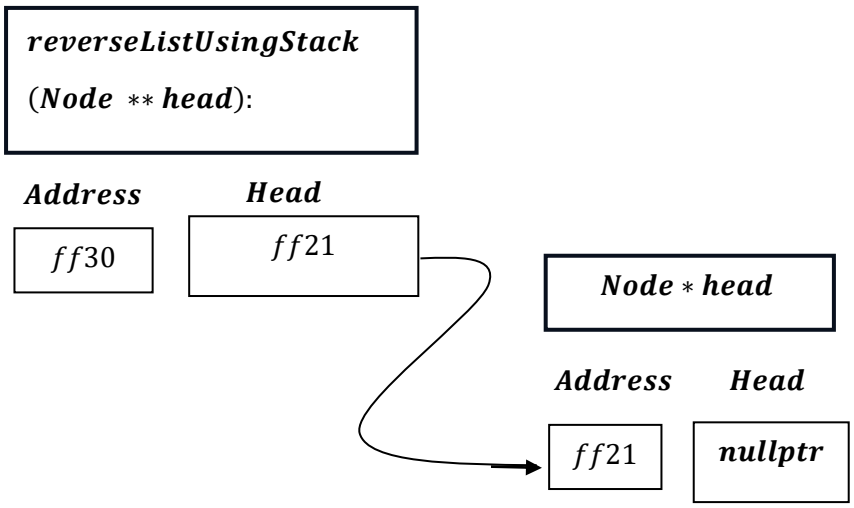
Now, as we call the function :

```
            reverseListUsingStack(&head);
```

Say, reverseListUsingStack(&head: ff21);

But our list is Empty, hence :

```
void reverseListUsingStack(Node **head) {…}
```



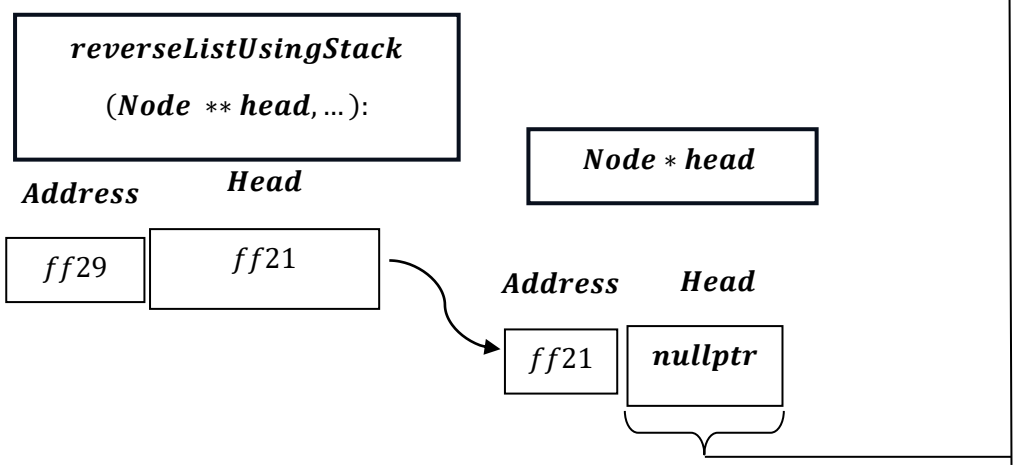
Now if Head pointer of Node structure points to nullptr then list is empty ,

i.e. Head carries value nullptr (not pointing to any valid memory location)

```
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
```

if(head == nullptr) if it is true*

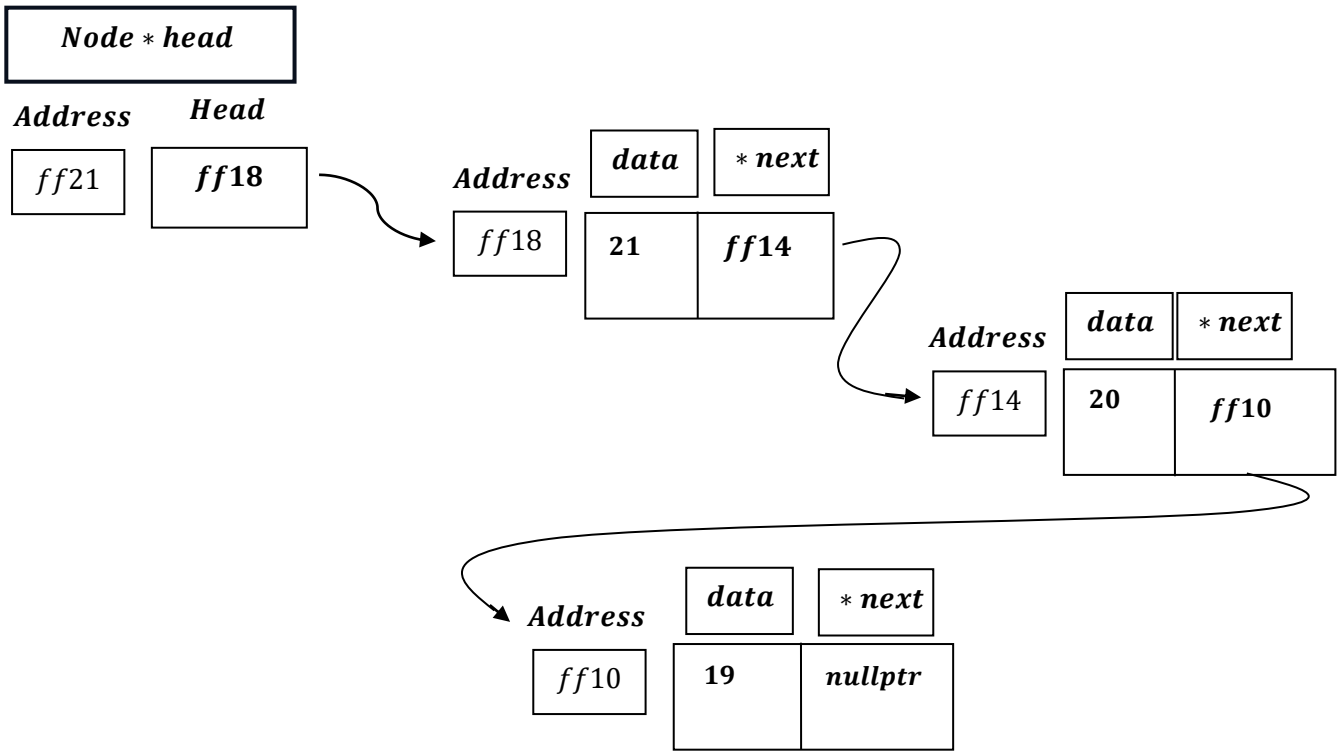
If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the reverseListUsingStack function.



Then , it prints a message : `Error: List is empty.` and return.

Now, List is not Empty

We are given with a Linked List:



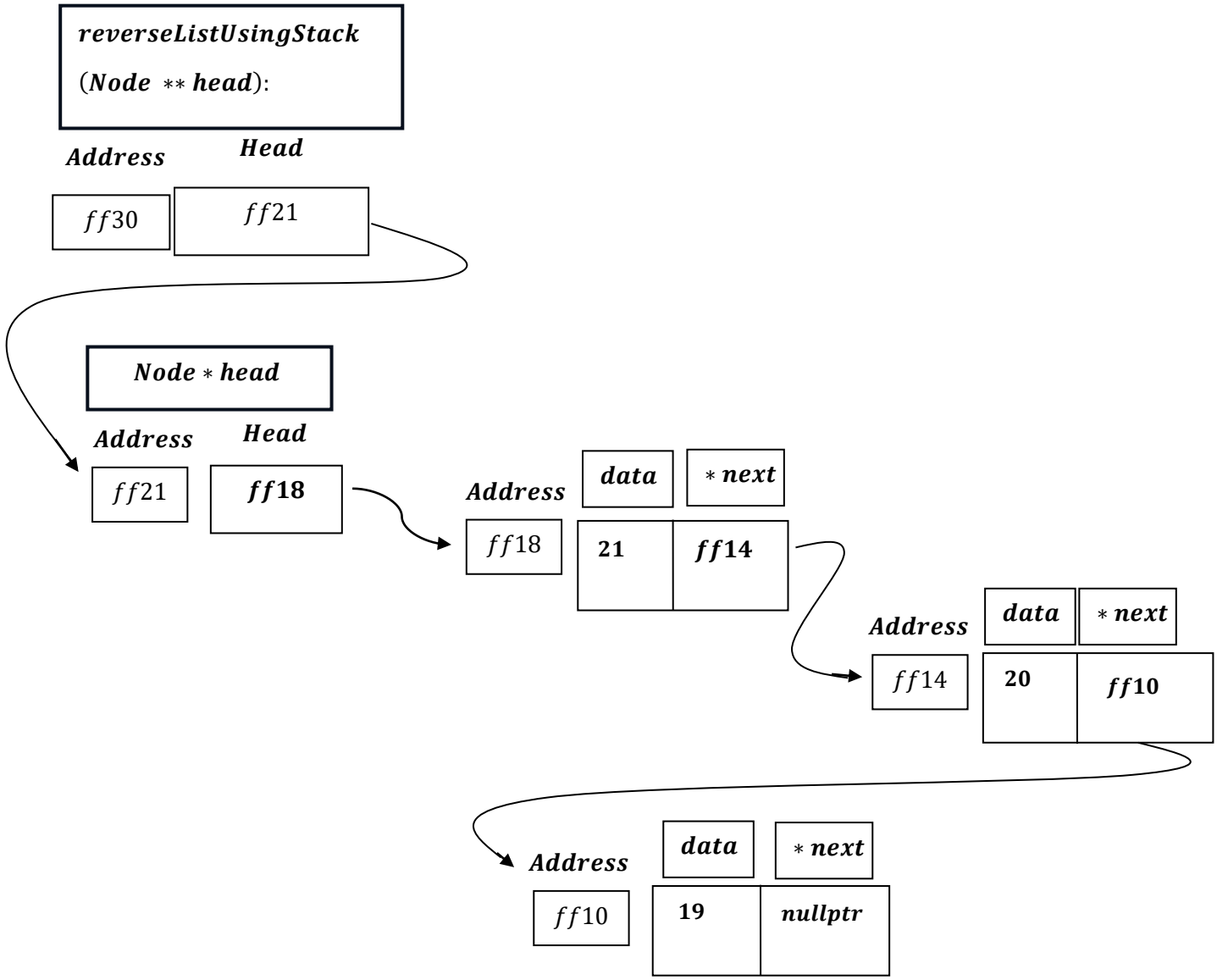
Next Function Call: ***reverseListUsingStack***(&***head***);

```
reverseListUsingStack(&head);
```

i.e.reverseListUsingStack(&***head***: ***ff21***);

Now:

```
void reverseListUsingStack(Node **head) {…}
```



Now , come to else part:

```
#include <stack>
….
else
    {
        stack<Node *> s;
        Node *current = *head;
        while (current != nullptr)
        {
            s.push(current);
```

```
            current = current->next;
        }
        *head = s.top();
        s.pop();
        current = *head;
        while (!s.empty())
        {
            current->next = s.top();
            s.pop();
            current = current->next;
        }
        current->next = nullptr;
    }
}
```

A brief introduction to `STL Library`

The Standard Template Library (STL) is a powerful set of C++ template classes to provide general-purpose classes and functions with templates that implement many popular and commonly used algorithms and data structures like vectors, lists, queues, and stacks. The STL has four components:

1. *Algorithms: These are a collection of functions that perform operations on containers. Examples include searching, sorting, and manipulating data. Some common algorithms are `sort`, `find`, `reverse`, and `accumulate`.*

2. *Containers: These are used to store collections of objects. The STL provides several types of containers, each optimized for specific kinds of operations. Some common containers are:*

– $Sequence\ Containers$: `$vector$`, `$deque$`, `$list$`, `$array$`, `$forward\_list$`.

– $Associative\ Containers$: `$set$`, `$multiset$`, `$map$`, `$multimap$`.

– $Unordered\ Associative\ Containers$: `$unordered\_set$`, `$unordered\_multiset$`, , `$unordered\_map$`, `$unordered\_multimap$`.

– $Container\ Adapters$: `$stack$`, `$queue$`, `$priority\_queue$`.

$We\ can\ either\ implement\ STL\ libraries\ using$:

1. $\#include\ < bits/stdc++.h >\ \ header\ file.$

2. $Or\ calling\ individual\ libraries\ like\ here$: $\#include < Stack >\ \ header\ file.$

$A\ Brief\ Introduction$: $\#include < bits/stdc++.h >$

$The\ line$ `$\#include\ < bits/stdc++.h >$` $is\ a\ non-standard\ header\ file\ that\ includes\ virtually\ all\ standard\ C++\ libraries.\ It\ is\ commonly\ used\ in\ competitive\ programming\ to\ save\ time\ by\ not\ having\ to\ include\ each\ library\ individually.\ However,\ it\ is\ not\ recommended\ for\ production\ code\ or\ larger\ projects\ for\ several\ reasons$:

1. $Portability$: The `$bits/stdc++.h$` $header\ is\ specific\ to\ GCC\ (GNU\ Compiler\ Collection)\ and\ may\ not\ be\ available\ on\ other\ compilers\ or\ platforms.$

2. $Compilation\ Time$: $Including\ all\ standard\ libraries\ can\ significantly\ increase\ compilation\ time,\ especially\ for\ large\ projects.$

3. $Code\ Clarity$: $It\ can\ make\ the\ code\ less\ readable\ and\ harder\ to\ understand\ which\ specific\ libraries\ are\ being\ used.$

Instead, it is better practice to include only the headers you need.

A Brief Introduction : #include < stack >

Stack library is a part of Standard Template Library (STL), which contains stack functions : push, pop and other operations.

Note: –We can also create a different header file with our own stack function code and include in our file like: #include "stack.h" .
But as we are provided with an inbuilt library , its better to use .

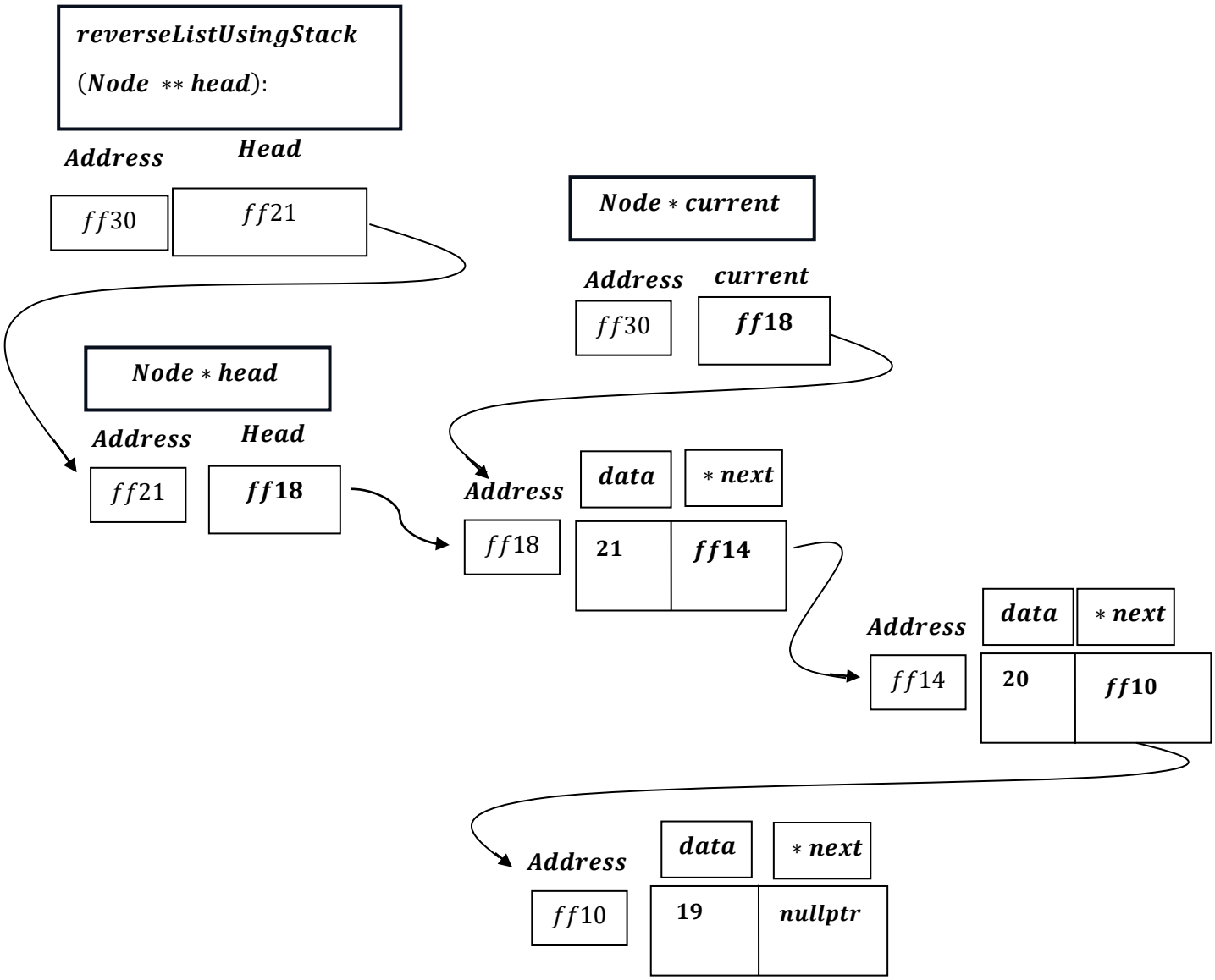
Now coming back to the program:

```
        stack<Node *> s;
```

Stack will hold a specific type : Node pointer variables and `s` is the Stack's object or say variable to deal with Stack operation.

Here `stack` of `stack < Node * > is class stack contains in stl_stack.h header file and hence `s` is object of the class stack.

```
        Node *current = *head;
```

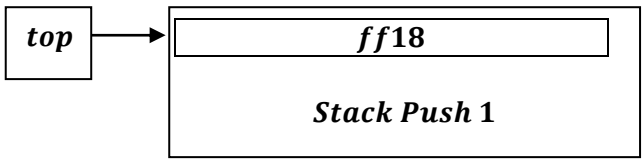


Next,

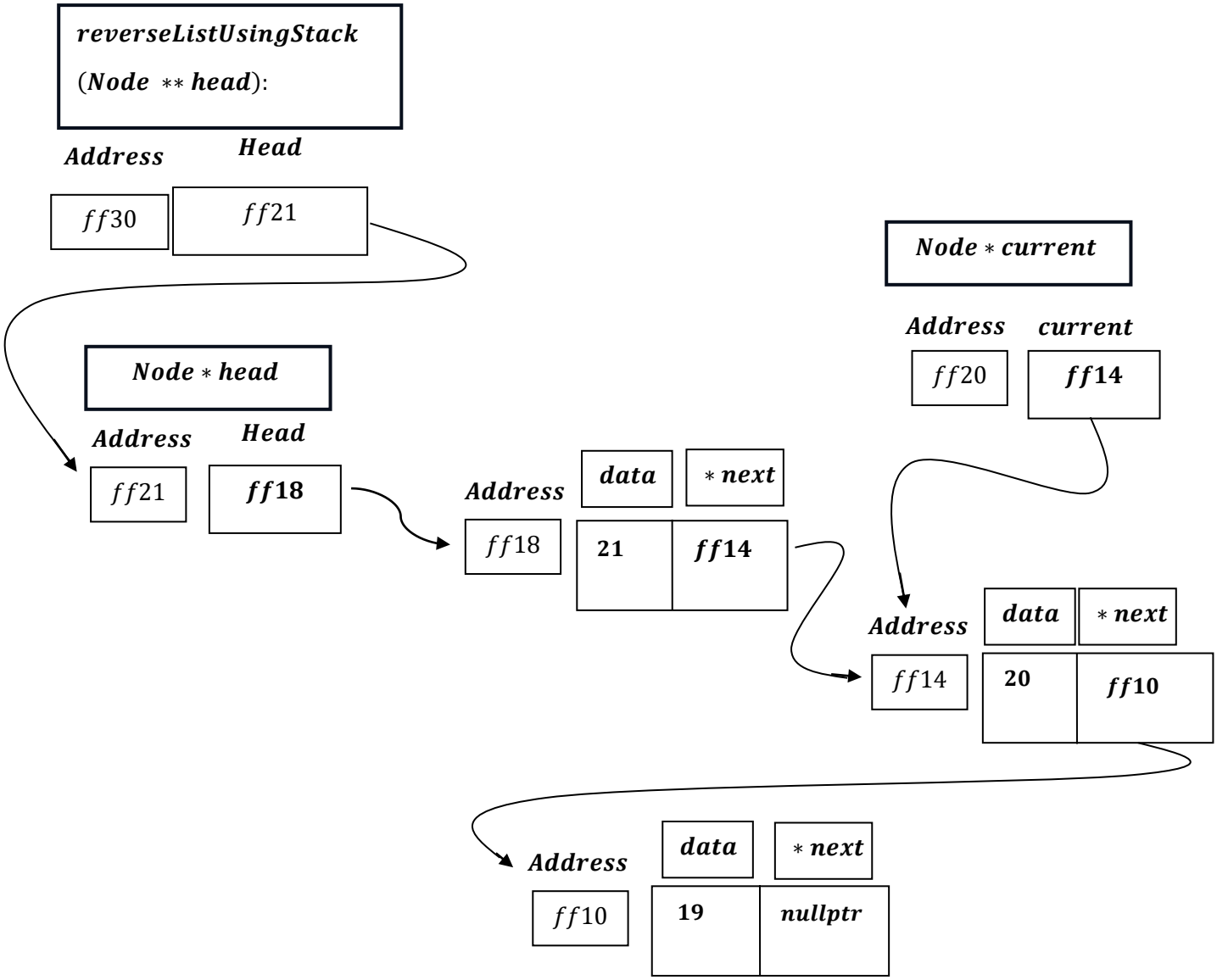
```
        while (current != nullptr)
        {
            s.push(current);
            current = current->next;
        }
```

Now entering into the while loop, current is : ff18 , not `nullptr`, then :

$\boldsymbol{Push\ 1}$:

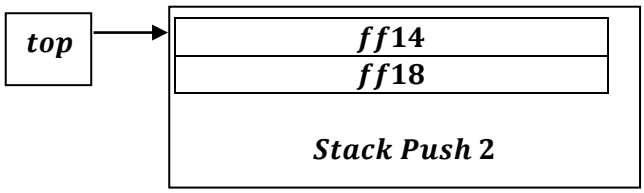


$\boldsymbol{Now, current = current \rightarrow next; hence\ current = ff14}$.

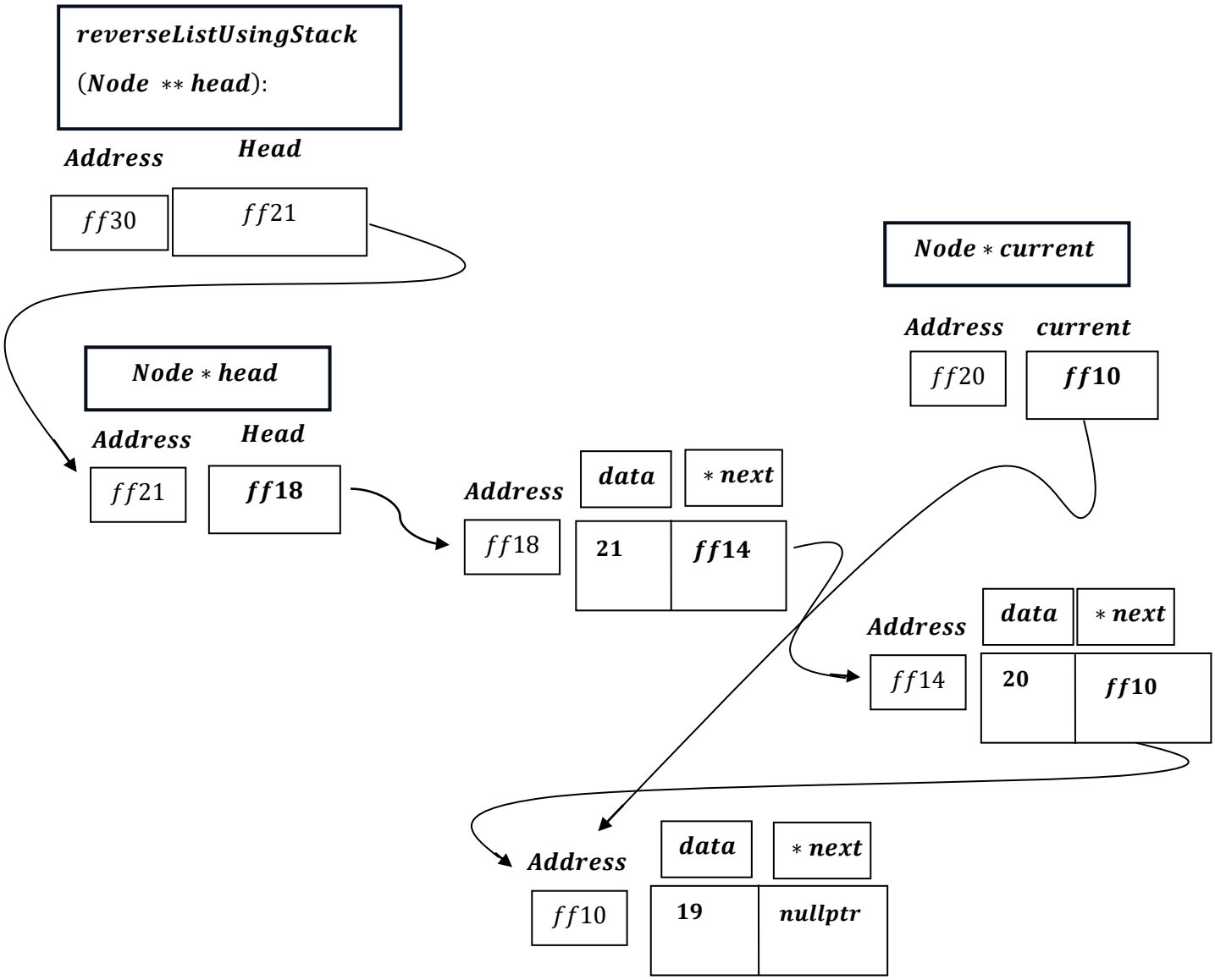


$Now\ , current\ is : ff14\ , not\ `nullptr`, then :$

$Push\ 2$:

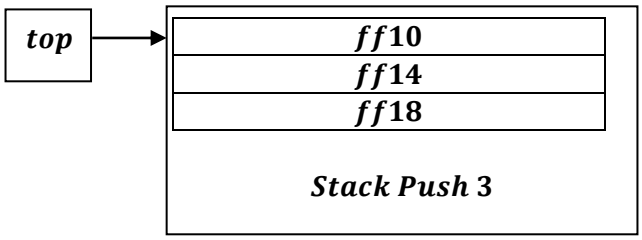


$Now, current = current \rightarrow next; hence\ current = ff10.$

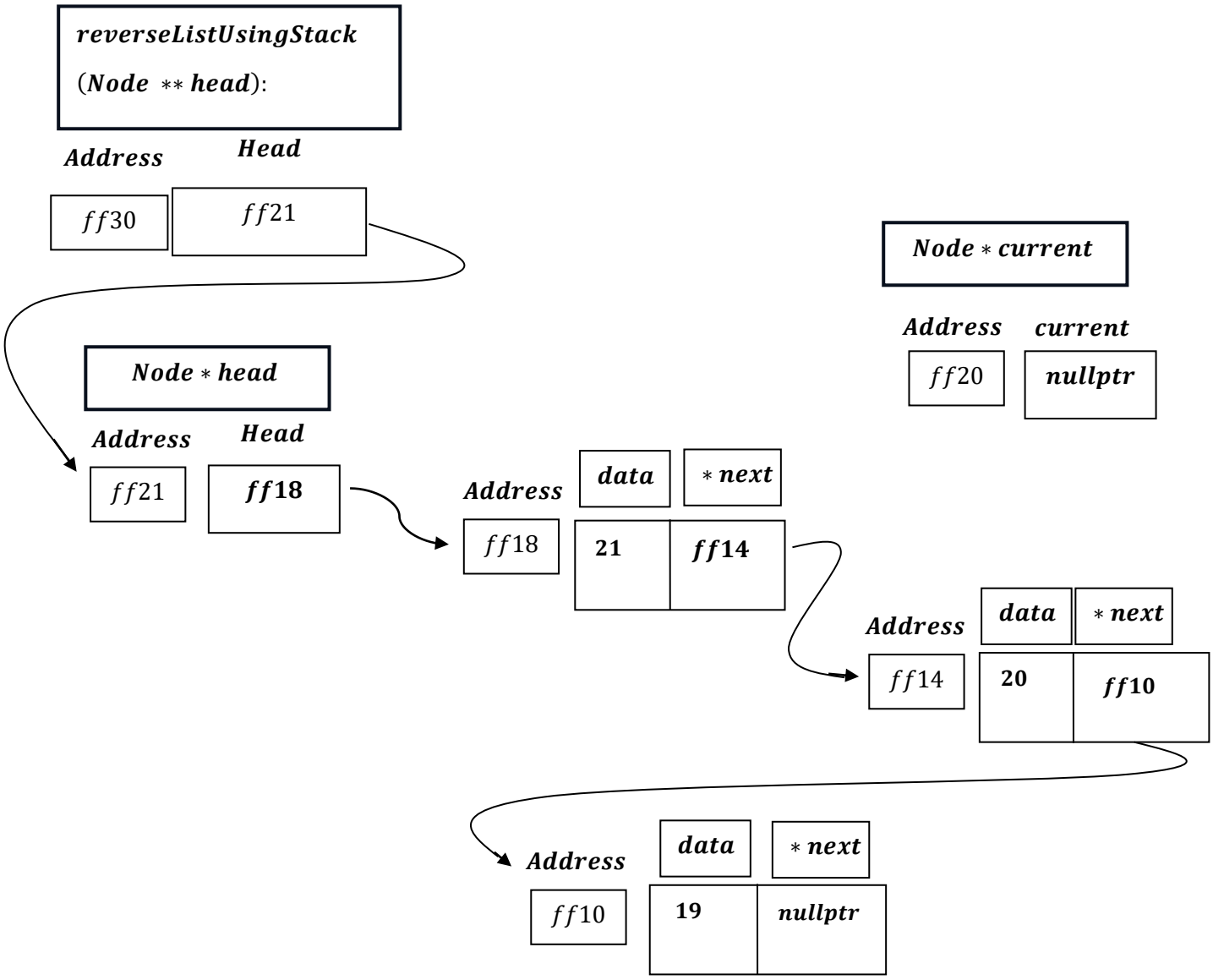


$Now\ , current\ is : ff10\ , not\ `nullptr`, then :$

$Push\ 3$:



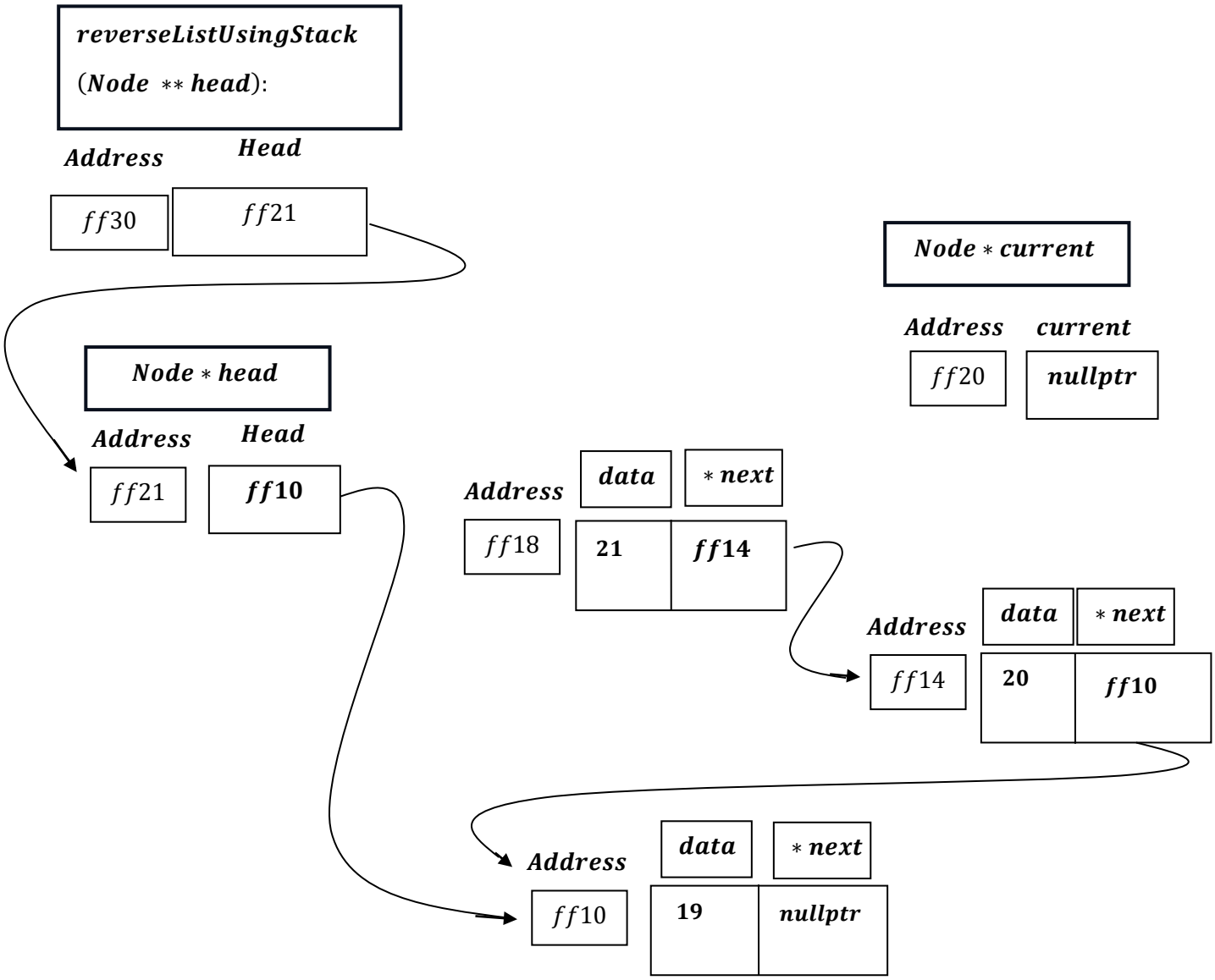
$Now, current = current \rightarrow next; hence\ current = `nullptr`.$



Now , current is : `nullptr` , hence loop exits .

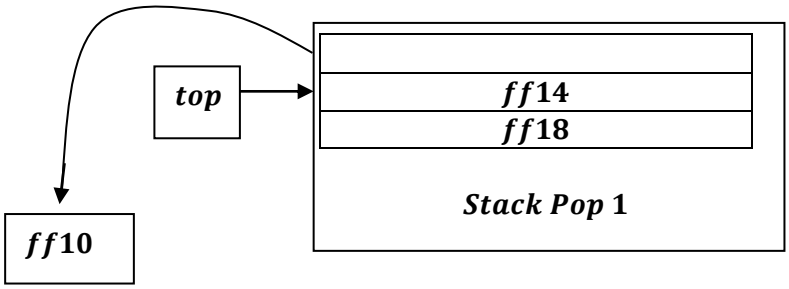
```
        *head = s.top();
```

i.e. $*$ *head* $= ff10$, *now*, *Node* $*$ *head will point to* $ff10$.



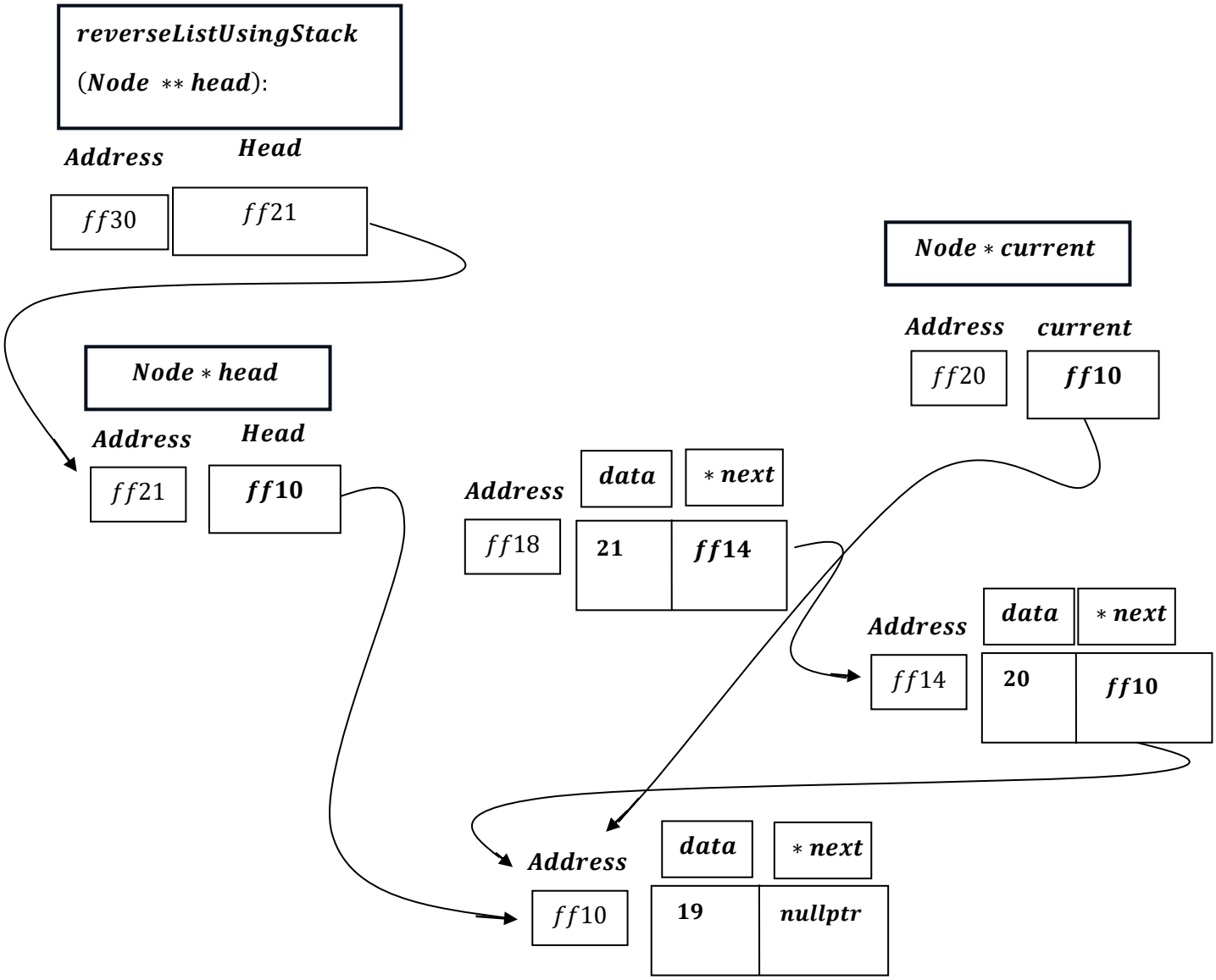
And , as now Node $*$ *head is already point to last node , hence no need of top of the stack which holds the address of last node , therefore pop out the top.*

Pop1:



Next,

```
        current = *head;
```



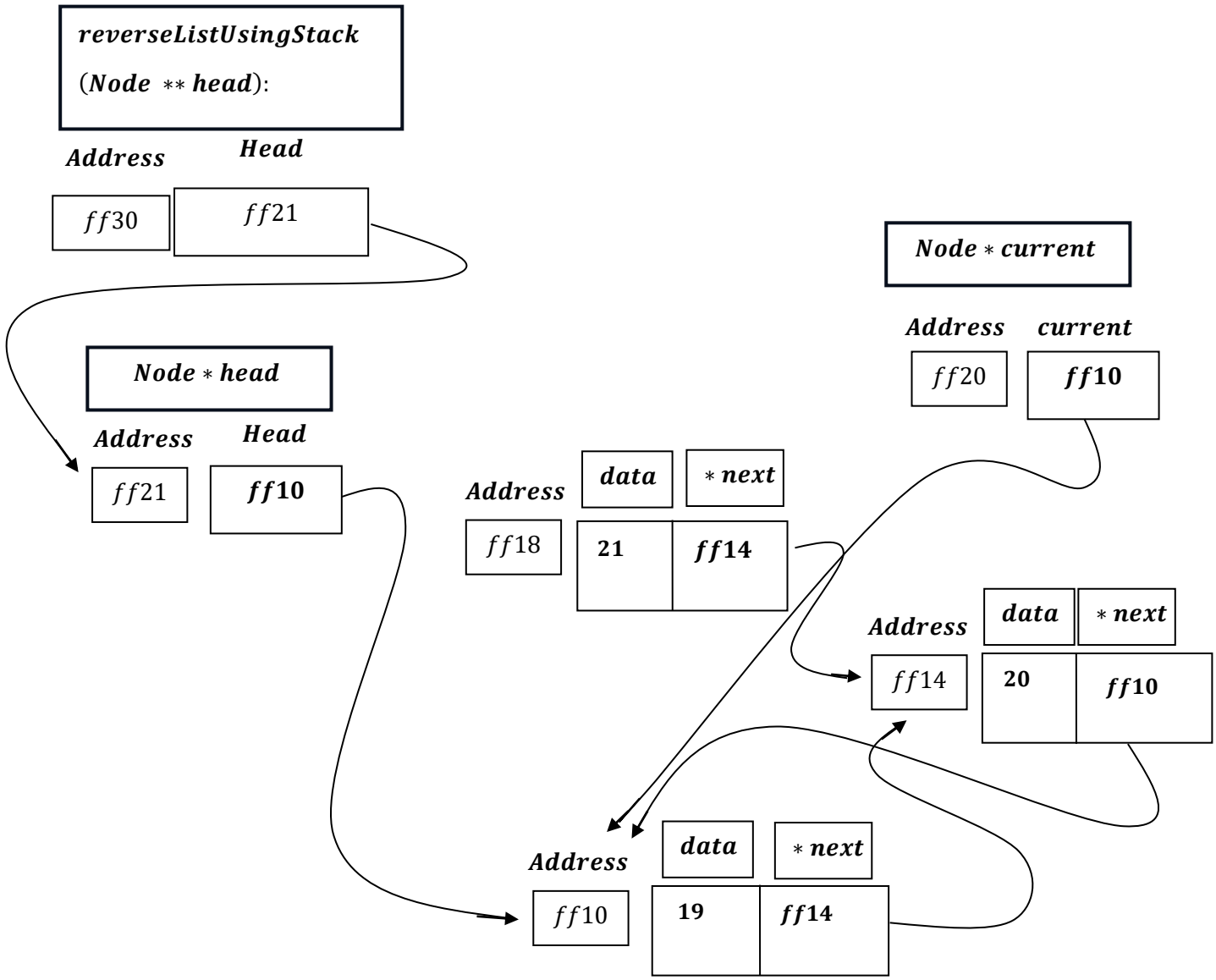
$Next,$

```
        while (!s.empty())
        {
            current->next = s.top();
            s.pop();
            current = current->next;
        }
        current->next = nullptr;
    }
```

$Therefore\ , now\ , again\ we\ enter\ in\ the\ while\ loop\ , and\ now\ Stack\ is\ not\ Empty,$ $hence$:

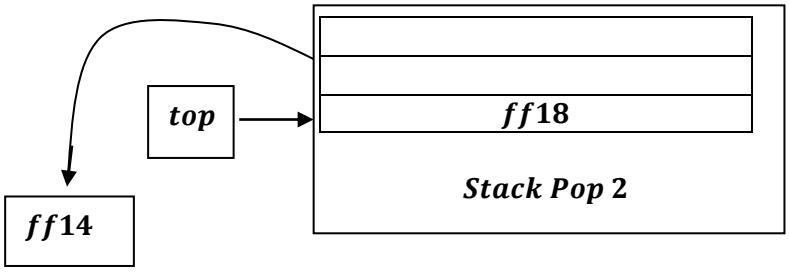
```
            current->next = s.top();
```

$current \rightarrow next = s.top() \ = ff14.$


reverseListUsingStack
(Node ** head):
Address
Head
ff30
ff21
Node * current
Address
current
ff20
ff10
Node * head
Address
Head
ff21
ff10
Address
data
* next
ff18
21
ff14
Address
data
* next
ff14
20
ff10
Address
data
* next
ff10
19
ff14

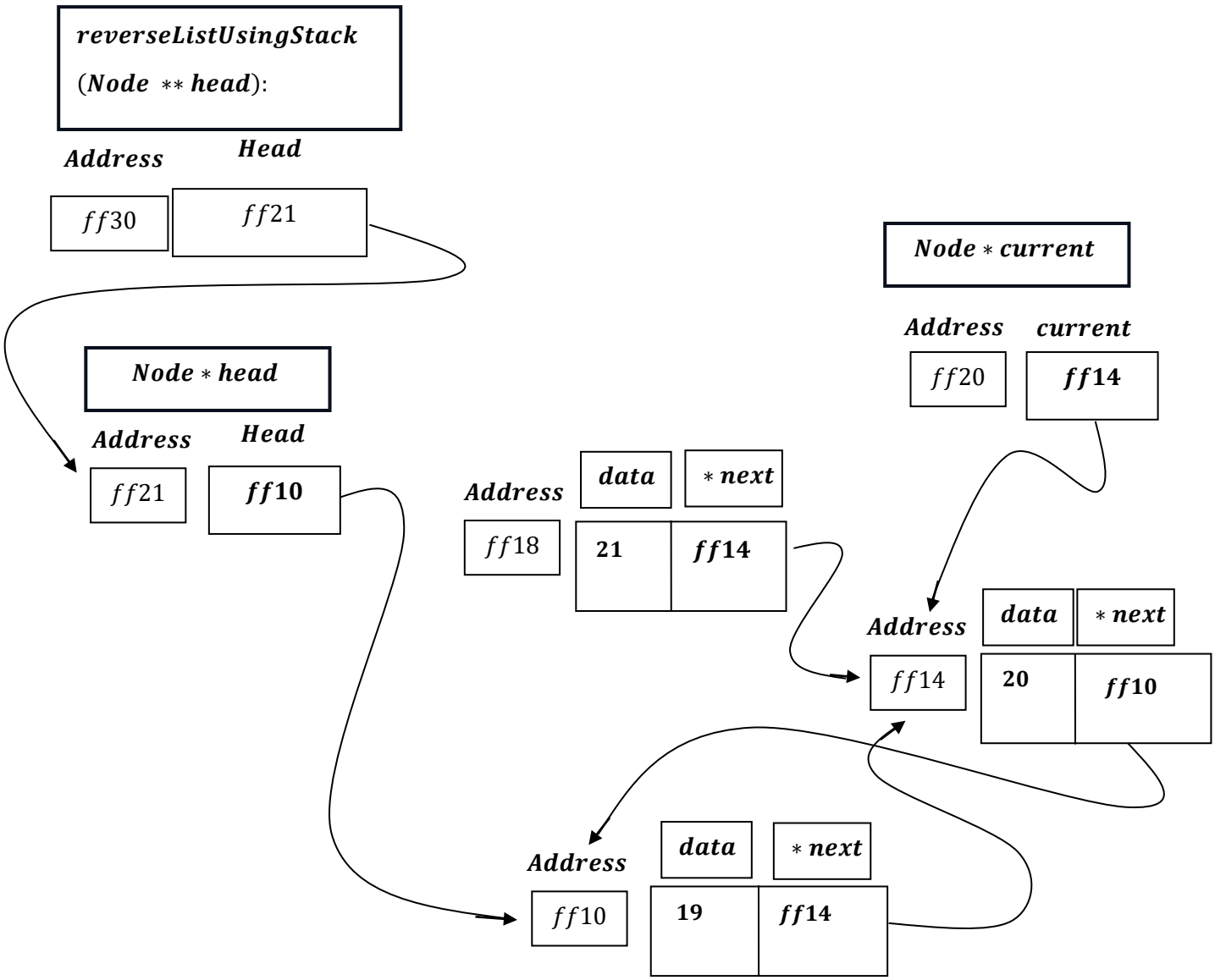
```
s.pop();
```

Pop 2:


top
ff18
Stack Pop 2
ff14

```
current = current->next;
```

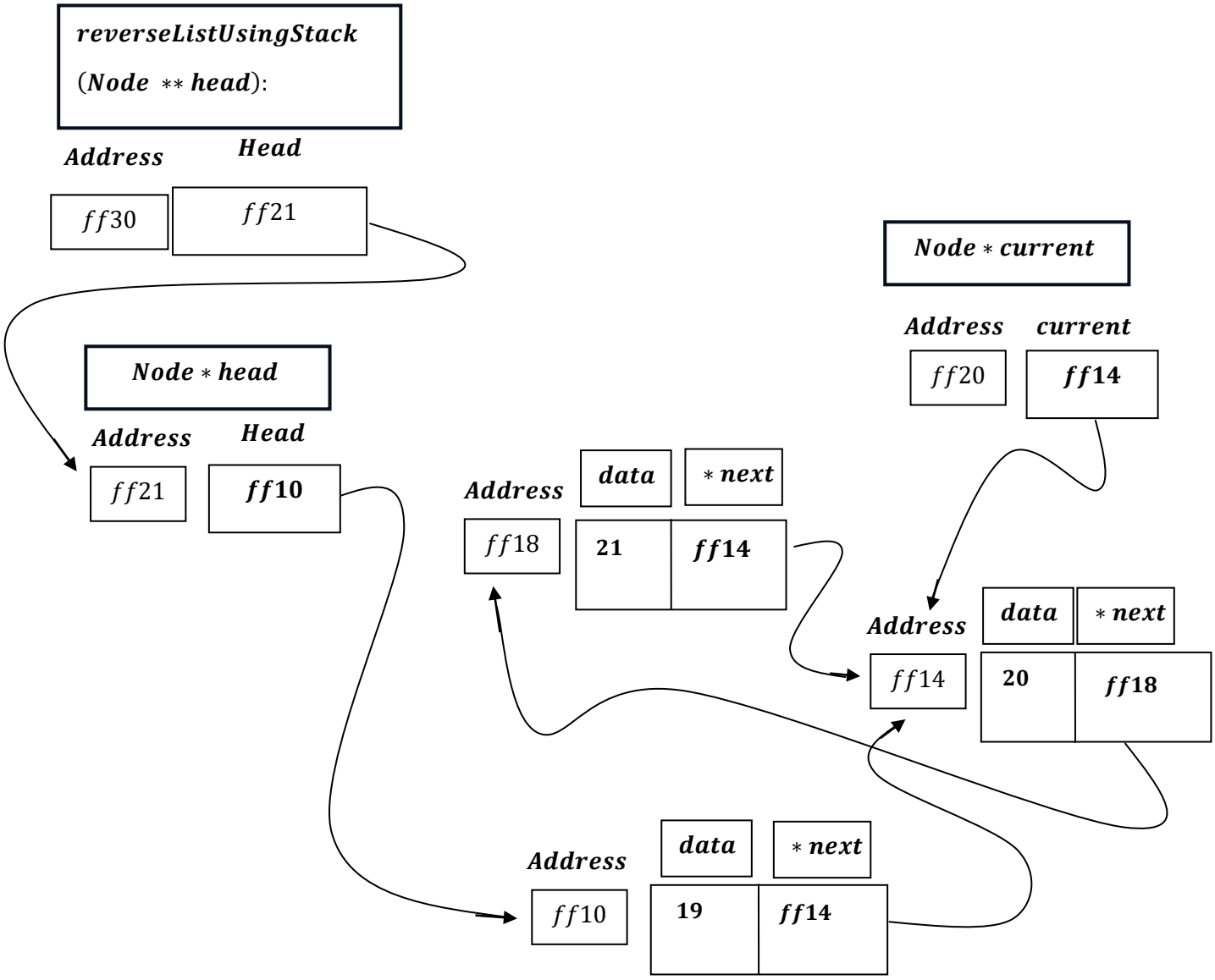
$current = current \rightarrow next; = ff14$.



Now , again , as Stack is not Empty, continuing with the loop, hence:

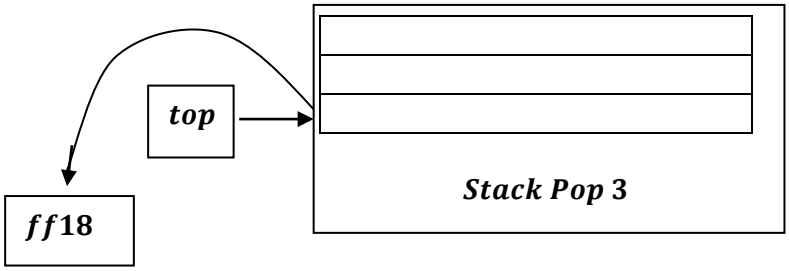
```
current->next = s.top();
```

$current \rightarrow next = s.top() \ = ff18.$



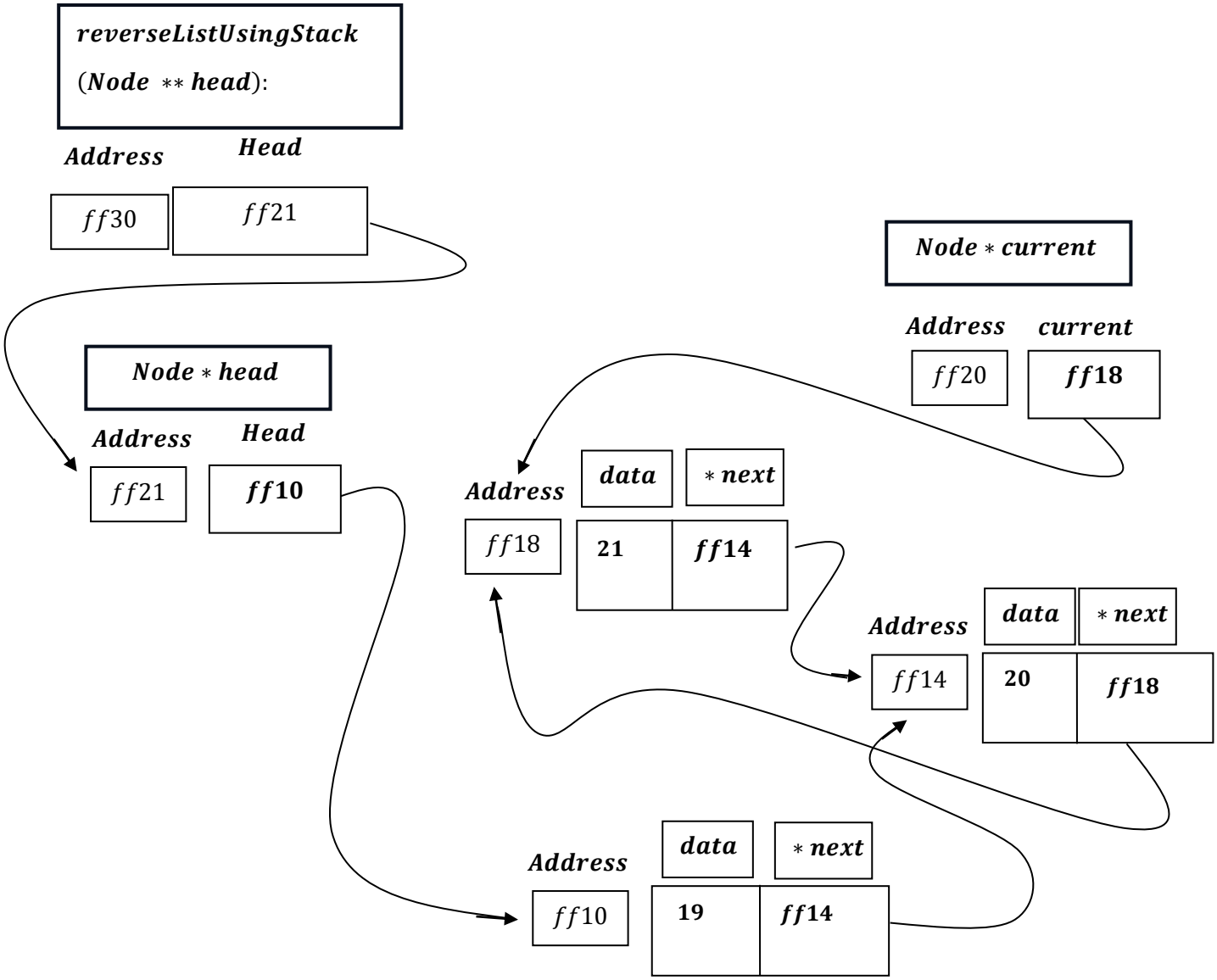

```
s.pop();
```

***Pop* 3**:



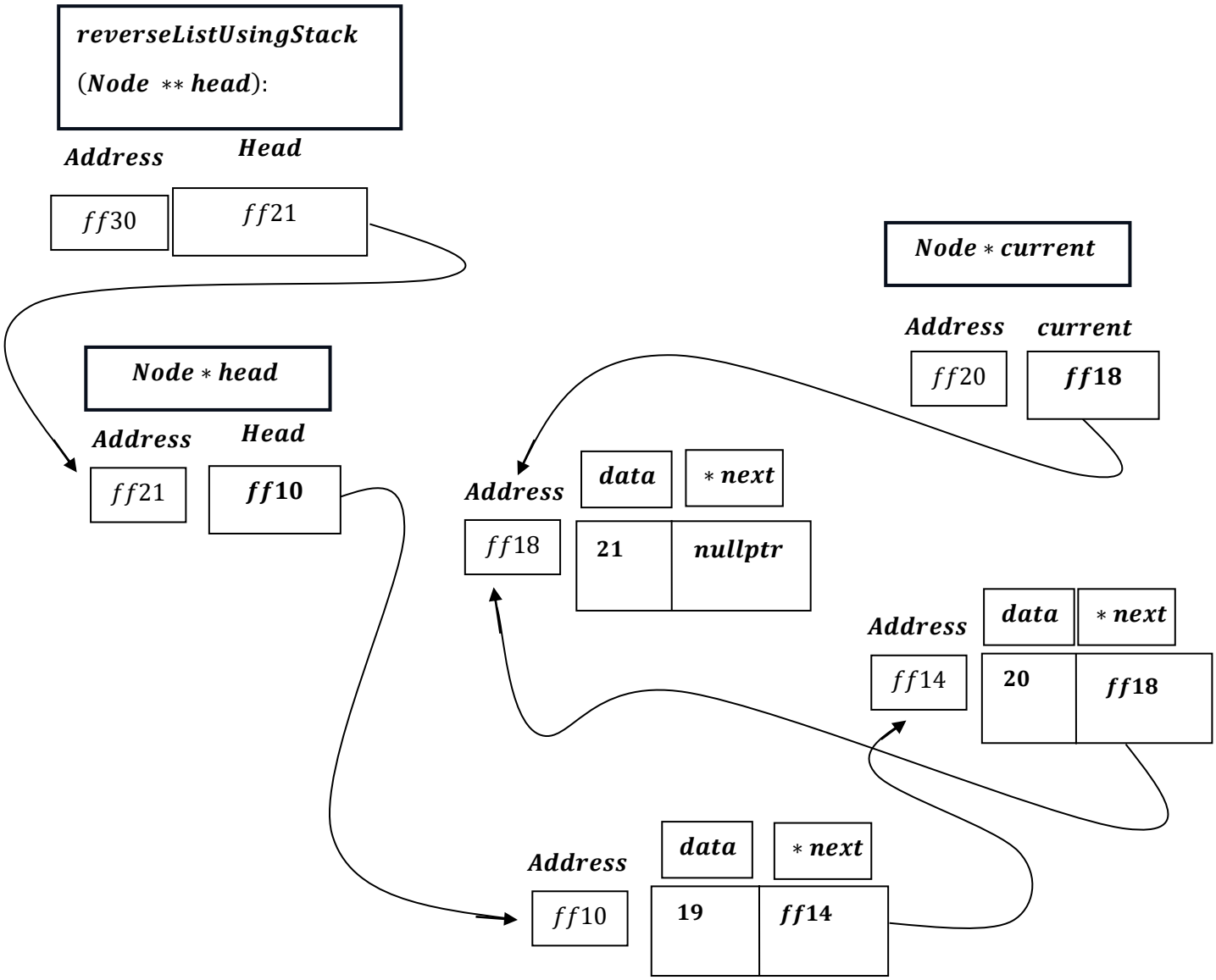

```
current = current->next;
```

$current = current \rightarrow next; = ff18$.

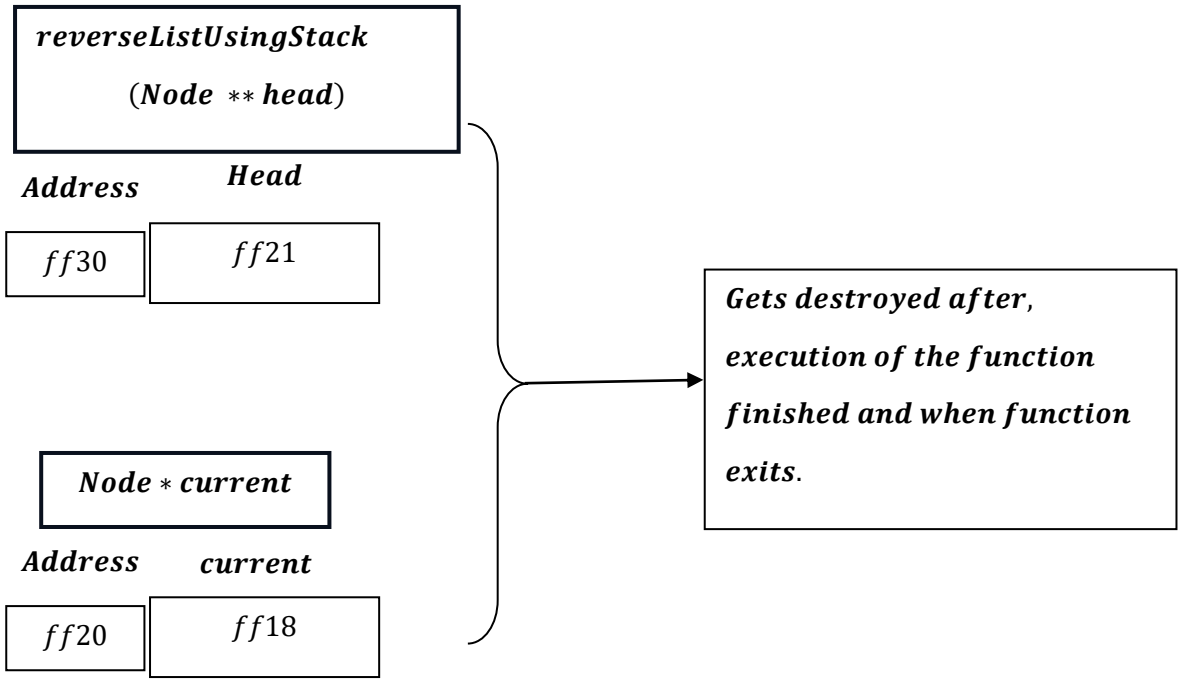


And now loop ends, as stack becomes empty.

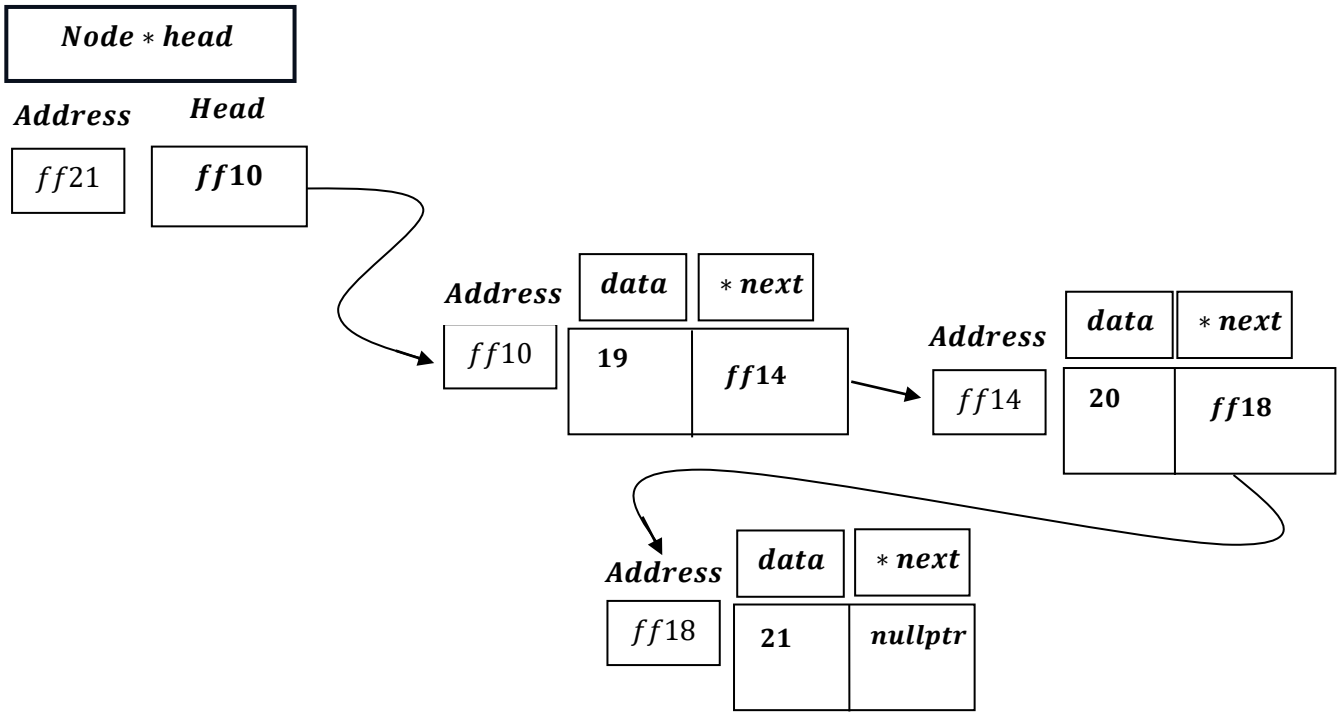
```
        current->next = nullptr;
```



Now as the function ends , all the temporary pointer variables gets destroyed:



And we are left with:



Step By Step explanation of reverseListUsingStack :

1. *Check if the List is Empty*:

- *The function first checks if the list is empty by checking if the head pointer is `nullptr`.*
- *If the list is empty, it prints an error message and returns immediately.*

2. *Initialize a Stack*

- *A stack `s` is initialized to hold the nodes of the linked list.*

3. *Push All Nodes onto the Stack*

- *A pointer `current` is initialized to point to the head of the list.*
- *The function then iterates through the entire list, pushing each node onto the stack.*
- *After pushing a node, `current` is moved to the next node in the list.*
- *This loop continues until `current` becomes `nullptr`, indicating the end of the list.*

5. *Reconstruct the List in Reverse Order*

- *The function then enters a loop that continues until the stack is empty.*
- *In each iteration, the next node of `current` is set to the top node of the stack.*
- *The top node is then popped from the stack.*
- *`current` is moved to this newly added node.*
- *This process effectively reverses the order of the nodes.*

6. *Terminate the New List*

- *After the stack is empty, `current` points to the last node of the reversed list.*
- *The `next` pointer of this node is set to `nullptr` to mark the end of the list.*

Time Complexity of Reverse List Using Stack

```
void reverseListUsingStack(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        stack<Node *> s;
        Node *current = *head;
        while (current != nullptr)
        {
            s.push(current);
            current = current->next;
        }
        *head = s.top();
        s.pop();
        current = *head;
        while (!s.empty())
        {
            current->next = s.top();
            s.pop();
            current = current->next;
        }
        current->next = nullptr;
    }
}
```

$O(1)$

$O(1)$

$O(1)$

runs`n` times at worst case .

runs $1 + 1 + \cdots n$ *times* $= O(n)$.

$O(1)$

$O(1)$

$O(1)$

runs`*$n - 1$*` times at worst case .

runs $1 + 1 + \cdots n - 1$ *times* $= O(n)$.

$O(1)$

$\underline{Best\ Case(When\ List\ is\ Empty)}$

$\rightarrow If\ the\ list\ is\ empty\ (`*head`\ is\ `nullptr`), print\ an\ error\ message\ and\ return\ from\ the\ function.$

$This\ takes\ O(1) time\ or\ \Omega(1) time\ .$

$\underline{Worst\ Case(When\ List\ is\ not\ Empty)}$

$\rightarrow If\ the\ list\ is\ empty\ (`*head`\ is\ `nullptr`), print\ an\ error\ message\ and\ return\ from\ the\ function. Though\ List\ is\ not\ empty\ here\ hence\ Checking\ takes\ O(1)\ constant\ time.$

$-As\ it\ moves\ to\ else\ part, stack < Node *> s\ , declaration\ takes\ O(1)\ time\ complexity.$

$-Current\ pointer\ of\ Node\ Structure's\ value\ gets\ assigned\ to\ Head\ pointer\ of\ Node\ structure\ takes\ contant\ amount\ of\ time\ i.e. O(1)\ time\ complexity.$

$-Now\ when\ we\ enter\ into\ the\ loop\ , till\ current\ pointer\ assigned\ to\ nullptr\ , the\ loop\ runs\ , with\ two\ loop\ inner\ statements:$

$s.push(current); \rightarrow\ \ runs\ `n`\ times.$
$current = current \rightarrow next; \rightarrow runs\ `n`\ times.$

$Hence\ total\ run\ is\ 2n\ and\ applying\ big\ O\ we\ get: O(2n) = O(n).$

$-Head\ pointer\ is\ assigned\ to\ stack's\ top\ and\ this\ operation\ takes : O(1).$

$-Popping\ stack\ s.pop\ ()\ takes\ constant\ amount\ of\ time : O(1).$

$-Current\ pointer\ of\ Node\ Structure's\ value\ gets\ assigned\ to\ Head\ pointer\ of\ Node\ structure\ takes\ contant\ amount\ of\ time\ i.e. O(1)\ time\ complexity.$

$-Now, again\ we\ get\ in\ to\ the\ loop\ till\ stack\ gets\ empty\ , the\ loop\ runs : n-1\ times, then$
$loop\ inner\ statements$:

$-current\ pointer\ points\ to\ next\ pointer\ of\ Node\ structure\ gets\ assigned\ to\ top\ of\ the\ stack$
$and\ this\ runs : 1+1+\cdots n-1\ times = n-1$.

$-Popping\ stack\ s.pop\ ()\ takes\ constant\ amount\ of\ time\ and\ total\ time\ the\ popping$
$occurs : 1+1+\cdots n-1\ times = n-1$.

$-current\ pointer\ 's\ \ value\ is\ assigned\ to\ next\ pointer\ of\ Node\ Structure\ pointed\ by$
$current\ pointer\ and\ this\ operation\ also\ runs :\ 1+1+\cdots n-1\ \ times = n-1.$

$Therefore\ , total\ time\ complexity : 3(n-1) = 3n-3\ and\ applying\ Big\ O\ , we\ get$:
$O(3n-3),$
$= 3 \times O(n-1)$
$= O(n-1)$
$= O(n) - O(1)$
$= O(n)\ time\ complexity.$

$And, next\ after\ the\ loop\ ends, current \rightarrow next\ is\ set\ to\ `nullptr`\ takes\ constant\ time$
$i.e.\ \ O(1)$.

$Therefore\ total\ time\ complexity\ is : O(1)+O(1)+O(1)+O(n)+O(1)+O(1)+O(1)+O(n)+$
$O(1) = O(n).$

$Hence\ , Worst\ Case\ time\ complexity\ is : O(n).$

$Average\ Case$

$We\ can\ say\ , if\ List\ is\ Empty\ , we\ get\ our\ best\ case\ scenario, and$
$when\ list\ is\ not\ empty\ , we\ get\ our\ worst\ case\ scenario. But\ though$

$best\ case\ and\ worst\ case\ scenarios\ are\ present\ as, always\ we$
$can\ determine\ list\ get\ traversed\ from\ beginning\ to\ end\ , hence\ \ we\ can\ say$
$\ \ average\ case\ is\ our\ worst\ case\ here\ i.e. always\ the\ entire\ list\ is\ traversed$
$i.e.\ \ O(n) = \Theta(n)\ .$

$This\ means\ the\ execution\ time\ is\ solely\ dependent\ on\ the\ length\ of\ the\ list.$
$If\ the\ list\ is\ empty, the\ execution\ time\ is\ constant\ (best\ case).$
$If\ the\ list\ is\ not\ empty, the\ execution\ time\ is\ proportional\ to\ the$
$length\ of\ the\ list\ (worst\ case). Since\ the\ loop\ rotation\ is\ predetermined,$
$there's\ no\ element\ of\ randomness\ or\ varying\ input\ data\ to\ consider.$
$Therefore, the\ concept\ of\ an\ average\ case, where\ we\ average\ the\ execution$
$time\ across\ different\ input\ possibilities, doesn't\ strictly\ apply\ here.$
$Hence\ Reverse\ List\ Using\ Stack\ doesn't\ have\ a\ true\ average\ case.$

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:$
$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$
$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$

$Singly\ Linked\ List$	$Cases$	$Time\ Complexity$
$Reverse\ List\ Using\ Stack$	$1. Best\ Case$	$\Omega(1)$
	$2. Worst\ Case$	$O(n)$
	$3. Average\ Case\ (=\ Worst\ Case)$	$\Theta(n)$